Flyway — schema bazei, scrisă de tine

Sebastian · exerciții: migrări versionate în locul lui `ddl-auto: update`

Pornire

- Proiectul e cel din One Model: `Product`, `Category`, `ProductRepository` și `ViewProduct`, care salvează 3 produse și afișează `Produse: 3`.
- Pornești MySQL cu `docker compose up -d`, apoi aplicația cu `mvn spring-boot:run`. Acum `ddl-auto: update` creează tabela `products` în locul tău.
- O interogare „în MySQL" o rulezi din Git Bash, din folderul proiectului:

```
docker compose exec -T mysql mysql -uroot -proot products_db -e "SHOW CREATE TABLE products\G"
```

- Rulezi o dată starter-ul și salvezi ieșirea comenzii de mai sus: e tabela pe care o vei scrie singur la exercițiul 3.
- Fiecare exercițiu se încheie cu un commit `ex N`, pușat.

1 · Flyway pe baza veche

Noțiuni noi: `spring-boot-starter-flyway` `flyway-mysql` `ddl-auto: validate`

Faci:

- În `pom.xml` adaugi `org.springframework.boot:spring-boot-starter-flyway` și `org.flywaydb:flyway-mysql`, fără versiune.
- În `application.yaml`: `ddl-auto: validate`, iar la `logging.level` adaugi `org.flywaydb: info`.
- Pornești aplicația pe baza de acum, cea creată de Hibernate.

Gata când:

Aplicația se oprește cu:

```
Found non-empty schema(s) `products_db` but no schema history table.
Use baseline() or set baselineOnMigrate to true to initialize the schema history table.
```

Flyway rulează înaintea lui Hibernate și refuză o bază pe care n-a creat-o el.

2 · Bază nouă, fără migrări

Noțiuni noi: ce face `validate`

Faci:

- Ștergi baza cu `docker compose down -v`, o pornești din nou cu `docker compose up -d`, apoi pornești aplicația.

Gata când:

```
No migrations found. Are your locations set up correctly?
Schema validation: missing table [products]
```

Flyway n-are ce aplica, iar Hibernate doar verifică tabela: n-o mai creează.

3 · Prima migrare

Noțiuni noi: `db/migration` `V1__nume.sql` `flyway_schema_history`

Faci:

- `src/main/resources/db/migration/V1__create_products.sql`: un `CREATE TABLE products` cu toate coloanele din `Product`, după ieșirea salvată la Pornire.

- `category` o scrii `VARCHAR(20)`, cum cere `@Column(length = 20)`, nu `enum(...)`, cum o generase Hibernate.
- Constrângerile le dai cu nume: `uk_products_sku` pentru `UNIQUE (sku)` și `chk_products_stock` pentru `CHECK (stock >= 0)`.
- Pornești aplicația de două ori.

Gata când:

Prima rulare:

```
Migrating schema `products_db` to version "1 - create products"
Produse: 3
```

A doua rulare: `Schema `products_db` is up to date. No migration necessary.`, apoi `Produse: 3`.

În MySQL, `SELECT version, description, success FROM flyway_schema_history` dă un singur rând: `1 | create products | 1`.

4 · Câmp nou, fără migrare

Noțiuni noi: entitatea și tabela trebuie să se potrivească

Faci:

- În `Product` adaugi `private String description;` cu `@Size(max = 255, message = "Description must be at most 255 characters")`, opțional. Nu scrii nicio migrare.

Gata când:

```
Schema validation: missing column [description] in table [products]
```

5 · A doua migrare

Noțiuni noi: `ALTER TABLE ... ADD COLUMN` · o migrare rulată nu se mai editează

Faci:

- Întâi creezi fișierul cu un singur `_`: `V2_add_product_description.sql`, cu `ALTER TABLE products ADD COLUMN description VARCHAR(255) NULL;`, și pornești aplicația.
- Apoi îl redenumești în `V2__add_product_description.sql` și pornești din nou.

Gata când:

Cu un singur `_`: aceeași eroare ca la 4. Flyway ignoră fișierul, fără niciun avertisment.

Cu `__`:

```
Migrating schema `products_db` to version "2 - add product description"
Produse: 3
```

În MySQL, `SHOW COLUMNS FROM products LIKE 'description'` arată `varchar(255)`, cu `Null = YES`.

Încearcă:

- În V1 schimbi `VARCHAR(100)` în `VARCHAR(120)`: aplicația nu mai pornește, cu `Migration checksum mismatch for migration version 1`. Pui V1 înapoi exact cum era.

6 · O bază veche, cu baseline

Noțiuni noi: `baseline-on-migrate` `baseline-version`

Situația reală: baza de producție a fost creată de Hibernate și nu poți s-o ștergi.

Faci:

- Refaci baza veche: `docker compose down -v`, `docker compose up -d`, apoi `git switch --detach <sha-ul primului commit>` (îl vezi cu `git log --oneline`), `mvn spring-boot:run` și `git switch -`.
- Pornești fără să schimbi nimic în `application.yaml`, doar prin variabile de mediu:

```
SPRING_FLYWAY_BASELINE_ON_MIGRATE=true SPRING_FLYWAY_BASELINE_VERSION=1 mvn spring-boot:run
```

Gata când:

În MySQL, `SELECT version, description, type FROM flyway_schema_history`:

```
1 | << Flyway Baseline >>  | BASELINE
2 | add product description | SQL
```

V1 n-a rulat: baseline-ul spune „baza e deja la versiunea 1”. V2 a rulat peste ea.

La final revii la baza ta: `docker compose down -v`, `docker compose up -d` și pornești normal. Cele două migrări rulează din nou ca la 3 și 5.

7 · O migrare care pică

Noțiuni noi: `RENAME COLUMN` · `success = 0` · repararea istoricului

Faci:

- `V3__rename_stock_to_quantity.sql` cu `ALTER TABLE products RENAME COLUMN stock TO quantity;`, iar în `Product`, pe `stock`, înlocuiești `@Column(nullable = false)` cu `@Column(name = "quantity", nullable = false)`. Câmpul Java rămâne `stock`.
- Pornești de două ori și citești ce s-a întâmplat.
- Repari: în MySQL `DELETE FROM flyway_schema_history WHERE success = 0`, iar V3 îl rescrii în 3 comenzi: scoți `chk_products_stock`, redenumești coloana, pui la loc verificarea ca `chk_products_quantity` pe `quantity`.

Gata când:

Prima rulare pică în MySQL:

```
Error Code : 3959
Message    : Check constraint 'chk_products_stock' uses column 'stock', hence column cannot
be dropped or renamed.
```

A doua rulare nici nu mai încearcă: `Detected failed migration to version 3 (rename stock to quantity)`.

După reparare, V3 rulează, apare `Produse: 3`, iar în `flyway_schema_history` versiunea 3 are `success = 1`.

V3 l-ai putut edita fiindcă n-a reușit niciodată. O migrare reușită nu se editează (exercițiul 5).

8 · Provocări

Fără indicii. Câte o migrare nouă pentru fiecare.

fiecare produs are un furnizor `supplier`, obligatoriu, fără câmp în `Product`; produsele salvate de aplicație primesc `unknown`	în MySQL `SELECT DISTINCT supplier FROM products` → `unknown`; `Produse: 3`
un nume de produs se poate repeta doar în categorii diferite; în `ViewProduct` salvezi încă un `Espresso`, tot în `COFFEE`, cu sku `COF-0009`	`Produse: 3`, apoi `Name already used in category: Espresso`